STAR Robot Control System

Comprehensive Technical Documentation

Complete Reference for Software Engineers

Version 1.0
December 2025

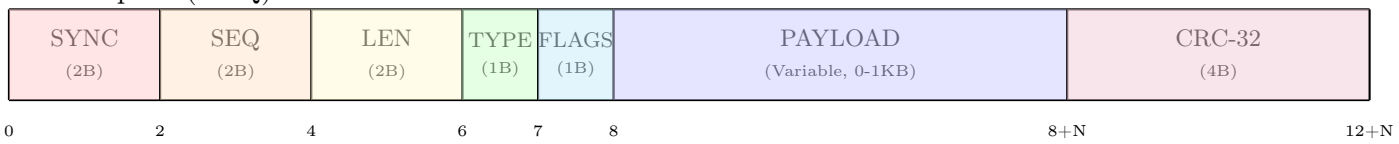
STAR Development Team
Locked Inc.

Contents

1 Nanopb (Protobuf) Protocol with ARQ and CRC for SPI Communication

1.1 Executive Summary

Overview: Protocol Buffers (nanopb) encapsulated in frames with CRC-32 and Automatic Repeat Request (ARQ) for reliable SPI communication between RPi5 and ESP32-S3.

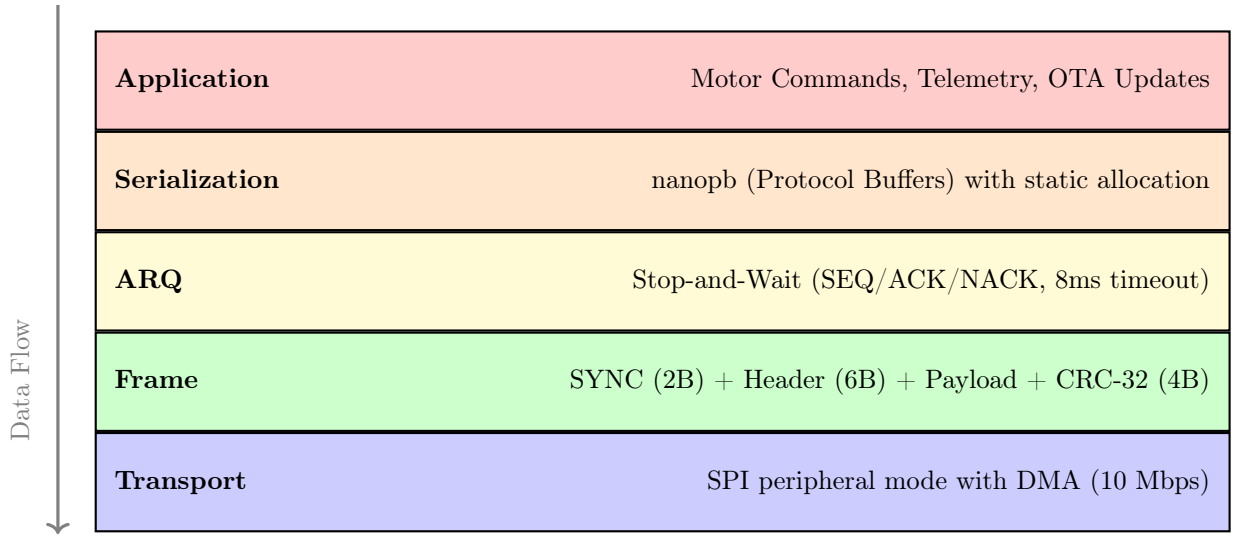


System Constraints:

- ESP32-S3-WROOM-1-N16: 16MB flash, 512KB SRAM, NO PSRAM
- 100Hz SPI communication (10ms period)
- 250Hz motor control loop (4ms period)
- Static allocation only (no malloc)

1.2 Key Design Decisions

1.2.1 Protocol Stack



1.2.2 Memory Budget

Table 1: SRAM Usage (512 KB total)

Component	Size (KB)	Usage (%)
DMA double-buffer	8.0	1.56
RX/TX buffers	2.0	0.39
Protobuf structs	1.5	0.29
ARQ state	4.5	0.88
Total	**16.0**	**3.12**

Table 2: Flash Usage (16 MB total)

Component	Size (KB)	Usage (%)
Generated protobuf	40	0.24
Nanopb runtime	12	0.07
Frame + ARQ layers	8	0.05
Total	**60**	**0.37**

Memory Budget Calculations:

SRAM (512 KB total):

- **DMA double-buffer:** 2×4092 bytes $= 8184$ bytes $= 8.0$ KB
  Calculation: Two ping-pong DMA buffers at `STAR_SPI_DMA_MAX_SIZE` (4092 bytes each)
  Usage: $\frac{8.0}{512} \times 100 = 1.56\%$
- **RX/TX buffers:** $1024 + 1024 = 2048$ bytes $= 2.0$ KB
  Calculation: 1 KB RX buffer + 1 KB TX buffer for staging protobuf messages
  Usage: $\frac{2.0}{512} \times 100 = 0.39\%$
- **Protobuf structs:** 1536 bytes $= 1.5$ KB
  Calculation: Largest message structures (SetVelocityResponse $\approx$ 600 bytes, TelemetryData $\approx$ 400 bytes)
  Usage: $\frac{1.5}{512} \times 100 = 0.29\%$
- **ARQ state:** $4 + 4 + 8 + 4 + 4092 = 4112$ bytes ≈ 4.5 KB
  Calculation: Sequence number (4B) + retry counter (4B) + timestamp (8B) + state (4B) + retry buffer (4092B)
  Usage: $\frac{4.5}{512} \times 100 = 0.88\%$
- **Total SRAM:** $8.0 + 2.0 + 1.5 + 4.5 = 16.0$ KB
  Total usage: $\frac{16.0}{512} \times 100 = 3.12\%$

Flash (16 MB = 16384 KB total):

- **Generated protobuf:** 40 KB
  Calculation: 3 proto files (common.proto, motor_control.proto, telemetry.proto) $\times$ 13 KB avg compiled size
  Usage: $\frac{40}{16384} \times 100 = 0.24\%$
- **Nanopb runtime:** 12 KB
  Calculation: Compiled size of pb_encode.c, pb_decode.c, pb_common.c
  Usage: $\frac{12}{16384} \times 100 = 0.07\%$
- **Frame + ARQ layers:** 8 KB
  Calculation: Framing logic + CRC-32 + ARQ state machine + retransmission logic
  Usage: $\frac{8}{16384} \times 100 = 0.05\%$
- **Total Flash:** $40 + 12 + 8 = 60$ KB
  Total usage: $\frac{60}{16384} \times 100 = 0.37\%$

1.3 Implementation Requirements

1.3.1 Protocol Stack Architecture

The protocol is implemented as a layered stack on both the RPi5 (SPI controller) and ESP32 (SPI peripheral). Each layer has specific responsibilities:

Layer	Responsibility	Implementation
5. Application	Motor commands, telemetry, control logic	Different per platform
4. Protobuf	Message encode/decode (nanopb)	Both sides
3. ARQ	Sequence numbering, ACK/NACK, retry logic	Both sides (symmetric)
2. Framing	Header/footer, CRC-32 verification	Both sides (identical)
1. SPI+DMA	Physical transport with double-buffering	Both sides (different roles)

Table 3: Protocol stack layers and implementation requirements

Key Insight: Layers 2-4 are nearly identical on both platforms. Only the application layer (Layer 5) and SPI role (Layer 1: controller vs peripheral) differ.

1.3.2 ESP32 (SPI Peripheral) Implementation

Layer 1: SPI Peripheral with DMA

- Configure SPI3 in peripheral mode using ESP-IDF
- Set up DMA with double-buffering: 2×4092 bytes
- Register DMA interrupt callbacks for buffer swapping
- Wait for chip select from RPi5, respond to clock

Layer 2: Frame Layer (Custom Code)

RX Path:

- Parse frame header (magic bytes, length, sequence number)
- Verify CRC-32 over header + payload
- If CRC fails $\rightarrow$ discard packet, send NACK
- If CRC valid $\rightarrow$ extract payload, pass to ARQ layer

TX Path:

- Build frame: `[Header | Payload | CRC32 | Footer]`
- Calculate CRC-32 over header + payload
- Queue complete frame to DMA buffer for transmission

Layer 3: ARQ Layer (Custom Code)

RX Path:

- Check sequence number against `last_acked_rx`
- If duplicate (`seq` $\leq$ `last_acked_rx`) $\rightarrow$ send ACK, discard payload
- If expected (`seq` = `last_acked_rx` + 1) $\rightarrow$ accept, send ACK
- If out-of-order $\rightarrow$ send NACK, request retransmission

TX Path:

- Assign `tx_sequence_number` to outgoing frame
- Store frame in retry buffer
- Start timeout timer (e.g., 10ms)
- On timeout $\rightarrow$ retransmit (up to 3 attempts)
- On ACK received $\rightarrow$ clear retry buffer, increment `tx_seq`

Layer 4: Protobuf (nanopb library)

- Decode incoming bytes $\rightarrow$ `SetVelocityRequest`, `EmergencyStopRequest`
- Encode outgoing structs $\rightarrow$ `SetVelocityResponse`, `TelemetryData`
- Use static allocation (no malloc) with `.options` file constraints

Layer 5: Application

- Handle motor velocity commands
- Read encoder data via multiplexer
- Execute PID control loop (250 Hz per motor)
- Send telemetry responses (encoder feedback, current, temperature)

1.3.3 RPi5 (SPI Controller) Implementation

Layer 1: SPI Controller with DMA

- Use Linux `/dev/spidev` interface
- Configure SPI3: 10 Mbps, mode 0, 8-bit words
- Use `ioctl(SPI_IOC_MESSAGE)` for DMA transfers
- Implement userspace double-buffering for continuous operation

Layer 2: Frame Layer *(Custom Code - SAME AS ESP32)*

RX Path: Parse frame header, verify CRC-32, extract payload or discard on CRC failure

TX Path: Build frame `[Header | Payload | CRC32 | Footer]`, calculate CRC-32, send via SPI

Layer 3: ARQ Layer *(Custom Code - SAME AS ESP32)*

RX Path: Check sequence number, detect duplicates $\rightarrow$ ACK and discard, detect out-of-order $\rightarrow$ NACK

TX Path: Assign sequence number, store in retry buffer, timeout and retry logic, handle ACKs

Layer 4: Protobuf (nanopb, protobuf-c, or gRPC)

- Encode: `SetVelocityRequest`, `EmergencyStopRequest`
- Decode: `SetVelocityResponse`, `TelemetryData`, `EncoderData`

Layer 5: Application

- Send motor velocity commands (from Nav2 or joystick)
- Receive encoder feedback for odometry
- Integrate with ROS2 navigation stack
- Handle UI commands and visualization

1.3.4 ARQ Symmetry and State Management

The ARQ protocol is **symmetric** — both sides implement identical logic but maintain separate state for their own transmissions and receptions.

Each Side Maintains:

- **TX sequence number** (`tx_seq`): For packets it sends
- **RX last acknowledged** (`rx_last_ack`): For packets it receives
- **Retry buffer**: Stores unacknowledged outgoing packets
- **Timeout timer**: Triggers retransmission if no ACK received

Responsibility	ESP32 (Peripheral)	RPi5 (Controller)
Sends	Telemetry, responses	Commands, requests
TX sequence numbering	Own `tx_seq`	Own `tx_seq`
ACK/NACK generation	Sends ACK for RPi5	Sends ACK for ESP32
Retry logic	Retries if no ACK	Retries if no ACK
Duplicate detection	Checks RPi5's `seq`	Checks ESP32's `seq`

Table 4: ARQ role distribution between controller and peripheral

Important: The SPI controller/peripheral roles (Layer 1) are **hardware-level only**. The ARQ layer (Layer 3) treats both sides as equals — each can send, ACK, and retry independently.

1.3.5 Example Transaction: RPi5 Sends Velocity Command

1. **RPi5 Application:** Create `SetVelocityRequest` (left=1.0 m/s, right=1.0 m/s)
2. **RPi5 Protobuf:** Encode to binary (nanopb)
3. **RPi5 ARQ:** Assign `seq=42`, save to retry buffer, start 10ms timeout
4. **RPi5 Frame:** Build packet with header, CRC-32, footer
5. **RPi5 SPI:** DMA transfer to ESP32 (controller initiates)
6. **ESP32 SPI:** DMA receives packet in double-buffer

7. **ESP32 Frame:** Verify CRC-32 → valid
8. **ESP32 ARQ:** Check `seq=42` = `rx_last_ack+1` → accept
9. **ESP32 Protobuf:** Decode to `SetVelocityRequest` struct
10. **ESP32 Application:** Update motor setpoints, execute PID control
11. **ESP32 ARQ:** Build ACK packet with `ack_seq=42`
12. **ESP32 Frame:** Wrap ACK in frame with CRC-32
13. **ESP32 SPI:** DMA transmits ACK (peripheral responds)
14. **RPi5 SPI:** DMA receives ACK packet
15. **RPi5 Frame:** Verify CRC-32 → valid
16. **RPi5 ARQ:** Got `ACK(42)` → clear retry buffer, cancel timeout

Outcome: Command successfully delivered with guaranteed delivery via ARQ. If ACK was lost or CRC failed, RPi5 would retransmit after 10ms timeout.

1.4 ARQ Communication Flow

1.4.1 Normal Operation (No Retransmission)

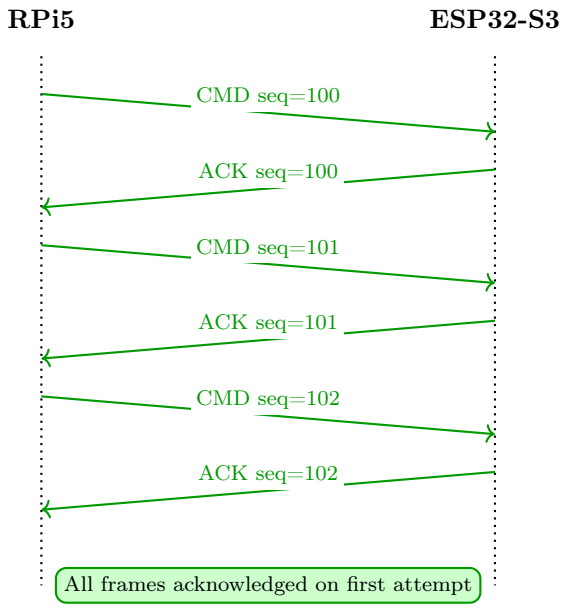


Figure 1: Normal ARQ Flow - Three Successful Transactions

1.4.2 Error Recovery Scenarios

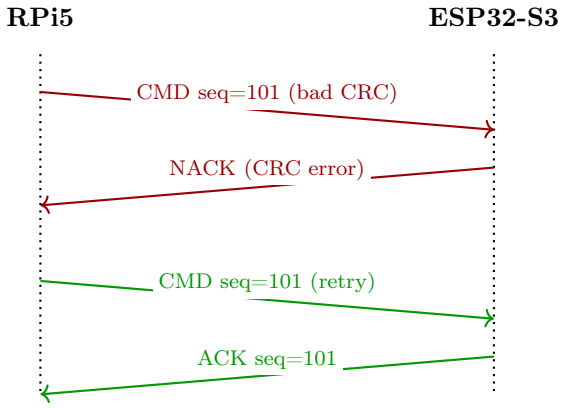


Figure 2: CRC Error - Retry with Same Sequence Number

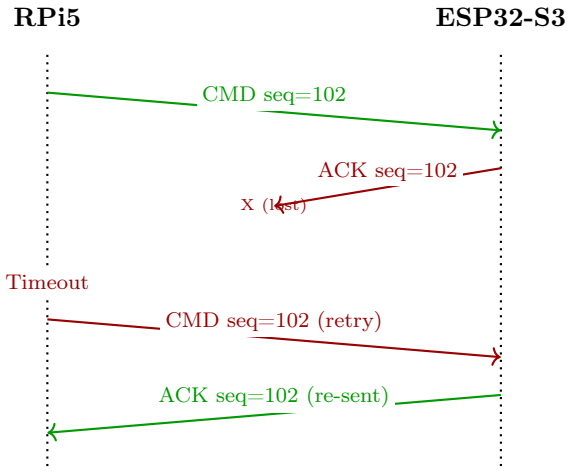


Figure 3: Duplicate Packet Scenario (Lost ACK)

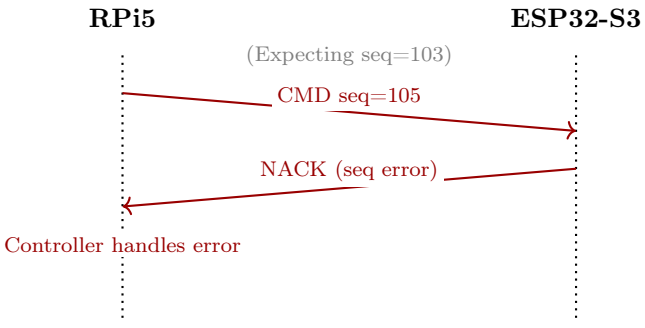


Figure 4: Out-of-Order Packet Scenario

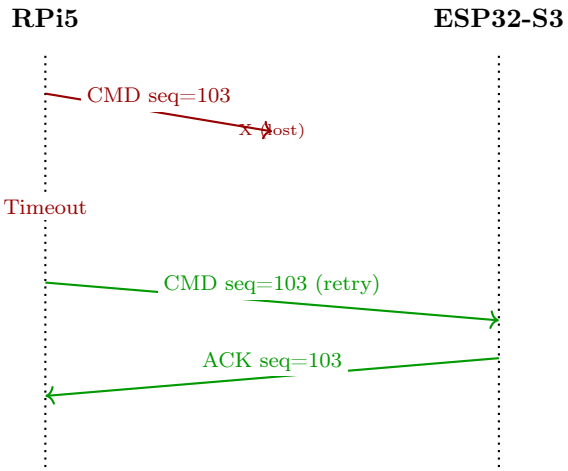


Figure 5: Lost Packet Scenario (Timeout)

1.5 Error Handling Strategies

CRC Mismatch
Action: NACK with CRC error reason
Recovery: Controller retry
Logging: Increment crc_errors

Sequence Error
Duplicate: Re-send ACK
Out-of-order: NACK + reject
Recovery: Controller retry

Timeout
RX timeout: Continue next cycle
500ms watchdog: Emergency stop
Retry fail: Log error

Invalid Protobuf
Action: NACK protocol error
Validate: IDs, ranges, offsets
Logging: Increment proto_errors

2 Hardware Pin Connections

2.1 ESP32-S3-WROOM-1-N16 Pinout

Critical Hardware Note: This module is the ESP32-S3-WROOM-1-**N16** variant (does NOT end with R_), which means it has **NO PSRAM**. We are allowed to use IO35-IO37 for SPI2 communication with motor drivers.

2.1.1 Main GPIO Assignments

Pin Number	GPIO Label	Signal Name / Function
27	IO0	GND
39	IO1	MOTOR_DRIVER_MOTOR1_IPROPI (ADC)
38	IO2	MOTOR_DRIVER_MOTOR2_IPROPI (ADC)
15	IO3	JTAG signal source strapping pin (NC)
4	IO4	MOTOR_DRIVER_MOTOR3_IPROPI (ADC)
5	IO5	MOTOR_DRIVER_MOTOR4_IPROPI (ADC)
6	IO6	MOTOR_DRIVER_MOTOR1_EN_IN1 (MCPWM)
7	IO7	MOTOR_DRIVER_MOTOR1_PH_IN2 (MCPWM)
12	IO8	FUEL_GUAGE_SMBD_SDA
17	IO9	FUEL_GUAGE_SMBC_SCL
18	IO10	ESP_PI_SPI3_PM_CS
19	IO11	ESP_PI_SPI3_PM_COPI
20	IO12	ESP_PI_SPI3_PM_SCLK
21	IO13	ESP_PI_SPI3_PM_CIPO
22	IO14	MOTOR_DRIVER_MOTOR2_EN_IN1 (MCPWM)
8	IO15	MOTOR_DRIVER_MOTOR2_PH_IN2 (MCPWM)
9	IO16	MOTOR_DRIVER_MOTOR3_EN_IN1 (MCPWM)
10	IO17	MOTOR_DRIVER_MOTOR3_PH_IN2 (MCPWM)
11	IO18	MOTOR_DRIVER_MOTOR4_EN_IN1 (MCPWM)
13	USB_D-	USB_C_N
14	USB_D+	USB_C_P
23	IO21	MOTOR_DRIVER_MOTOR4_PH_IN2 (MCPWM)
28	IO35	MOTOR_DRIVER_SPI2_CM_COPI
29	IO36	MOTOR_DRIVER_SPI2_CM_SCLK
30	IO37	MOTOR_DRIVER_SPI2_CM_CIPO
31	IO38	Decoder Input 1A0 (for Motor nCS selection)
32	IO39	Decoder Input 1A1 (for Motor nCS selection)
33	IO40	MOTOR_ENCODER_SEL0
34	IO41	MOTOR_ENCODER_SEL1
35	IO42	MOTOR_ENCODER_OUT0
26	IO45	Strapping pin for VDD_SPI voltage (NC)
16	IO46	Boot-mode / ROM-print strapping pin
24	IO47	MOTOR_ENCODER_OUT1
25	IO48	TEMPERATURE_SENSOR (DS18B20+)
37	TXD0	ESP_PI_SPI3_PM_DRDY1 (Interrupt to Pi)
36	RXD0	ESP_PI_SPI3_PM_DRDY2 (Interrupt to Pi)

Table 5: ESP32-S3 GPIO pin assignments

2.1.2 Pin Function Categories

SPI3 (Peripheral Mode - Communication with RPi5):

- IO10: Chip Select (CS)
- IO11: Controller Out Peripheral In (COPI)
- IO12: Serial Clock (SCLK)
- IO13: Controller In Peripheral Out (CIPO)
- TXD0 (IO1): Data Ready 1 interrupt to Pi
- RXD0 (IO3): Data Ready 2 interrupt to Pi

SPI2 (Controller Mode - Communication with Motor Drivers):

- IO35: Controller Out Peripheral In (COPI)
- IO36: Serial Clock (SCLK)
- IO37: Controller In Peripheral Out (CIPO)
- Chip selects generated via 74HC238 decoder (see below)

MCPWM (Motor Control):

- Motor 1: IO6 (EN), IO7 (PH)
- Motor 2: IO14 (EN), IO15 (PH)
- Motor 3: IO16 (EN), IO17 (PH)
- Motor 4: IO18 (EN), IO21 (PH)

ADC (Current Sensing - IPROPI):

- IO1: Motor 1 current
- IO2: Motor 2 current
- IO4: Motor 3 current
- IO5: Motor 4 current

Encoder Selection and Reading:

- IO40, IO41: Multiplexer select lines
- IO42, IO47: Encoder output channels

2.2 74HC238 Decoder (Motor Driver Chip Select)

The 74HC238 3-to-8 line decoder generates individual chip select signals for the four motor drivers using only two GPIO pins (IO38, IO39).

Pin Number	Pin Name	Signal Label / Connection	Source/Destination
1	$1\overline{E}$	GND (Always Enabled)	Ground
2	1A0	Connected to IO38	ESP32-S3
3	1A1	Connected to IO39	ESP32-S3
4	$1\overline{Y0}$	MOTOR_DRIVER_SPI2_MOTOR1_nCS	SPI2 Chip Select 1
5	$1\overline{Y1}$	MOTOR_DRIVER_SPI2_MOTOR2_nCS	SPI2 Chip Select 2
6	$1\overline{Y2}$	MOTOR_DRIVER_SPI2_MOTOR3_nCS	SPI2 Chip Select 3
7	$1\overline{Y3}$	MOTOR_DRIVER_SPI2_MOTOR4_nCS	SPI2 Chip Select 4
8	GND	GND	Ground
9-14	2Y3–2A0	NC (Not Connected)	Marked with "X"
15	$2\overline{E}$	P3V3_BUCK (Disabled)	Power Rail
16	VCC	P3V3_BUCK	Power Rail

Table 6: 74HC238 decoder pinout for motor driver chip select generation

Design Rationale: Using a hardware decoder reduces GPIO usage from 4 pins to 2 pins while providing active-low chip select signals required by the DRV8243S motor drivers.

2.3 74HC4052 Multiplexer (Encoder Channel Selection)

The 74HC4052 dual 4-to-1 multiplexer allows reading all four encoder channels (A and B) using only four GPIO pins instead of eight.

Pin Number	Pin Name	Signal Label / Connection	Source/Destination
1	$1\overline{G}$	GND (Always Enabled)	Ground
2	B	MOTOR_ENCODER_SEL1 (IO41)	ESP32-S3
3	1C3	MOTOR_DRIVER_ENCODER4_OUTPUT_A	Encoder 4
4	1C2	MOTOR_DRIVER_ENCODER3_OUTPUT_A	Encoder 3
5	1C1	MOTOR_DRIVER_ENCODER2_OUTPUT_A	Encoder 2
6	1C0	MOTOR_DRIVER_ENCODER1_OUTPUT_A	Encoder 1
7	1Y	MOTOR_ENCODER_OUT1 (IO47)	ESP32-S3
8	GND	GND	Ground
9	2Y	MOTOR_ENCODER_OUT0 (IO42)	ESP32-S3
10	2C0	MOTOR_DRIVER_ENCODER1_OUTPUT_B	Encoder 1
11	2C1	MOTOR_DRIVER_ENCODER2_OUTPUT_B	Encoder 2
12	2C2	MOTOR_DRIVER_ENCODER3_OUTPUT_B	Encoder 3
13	2C3	MOTOR_DRIVER_ENCODER4_OUTPUT_B	Encoder 4
14	A	MOTOR_ENCODER_SEL0 (IO40)	ESP32-S3
15	$2\overline{G}$	GND (Always Enabled)	Ground
16	VCC	P3V3_BUCK	Power Rail

Table 7: 74HC4052 multiplexer pinout for encoder channel selection

Design Rationale: The dual multiplexer allows simultaneous reading of both A and B channels from the selected encoder, reducing GPIO usage from 8 pins to 4 pins while maintaining quadrature decoding capability.

2.4 Raspberry Pi 5 Pinout

Pin	RPi Label	Signal Name / Connection	Status / Note
1	3v3	Not Connected	Pi 3v3 Rail (Not used)
2	5v	Not Connected	Pi 5v Rail (Not used)
3	SDA	PI_SDA	I2C Data
4	5v	Not Connected	Pi 5v Rail (Not used)
5	SCL	PI_SCL	I2C Clock
7	IO4	HCSR04_TRIG1	Ultrasonic Trigger 1
8	TXD	LiDAR_RX_PI5_TX	UART Transmit to LiDAR
9	GND	GND	Common Ground
10	RXD	LiDAR_TX_PI5_RX	UART Receive from LiDAR
11	IO17	HCSR04_ECHO1	Ultrasonic Echo 1
12	IO18	HCSR04_TRIG4	Ultrasonic Trigger 4
13	IO27	HCSR04_TRIG2	Ultrasonic Trigger 2
15	IO22	HCSR04_ECHO2	Ultrasonic Echo 2
16	IO23	IMU_RST	IMU Reset
17	3v3	Not Connected	Pi 3v3 Rail (Not used)
18	IO24	IMU_INT	IMU Interrupt
19	COPI	ESP_PI_SPI3_PM_COPI	SPI Controller Out
21	CIPO	ESP_PI_SPI3_PM_CIPO	SPI Controller In
22	IO25	HCSR04_ECHO4	Ultrasonic Echo 4
23	SCLK	ESP_PI_SPI3_PM_SCLK	SPI Clock
24	CE0	ESP_PI_SPI3_PM_CS	SPI Chip Enable 0
26	CE1	HCSR04_TRIG5	Ultrasonic Trigger 5
27	ID_SD	MOTOR_DRIVER_MOTOR4_nFAULT	Motor 4 Fault
28	ID_SC	HCSR04_ECHO5	Ultrasonic Echo 5
29	IO5	MOTOR_DRIVER_MOTOR3_nFAULT	Motor 3 Fault
31	IO6	MOTOR_DRIVER_MOTOR2_nFAULT	Motor 2 Fault
32	IO12	HCSR04_TRIG6	Ultrasonic Trigger 6
33	IO13	MOTOR_DRIVER_MOTOR1_nFAULT	Motor 1 Fault
35	IO19	HCSR04_TRIG3	Ultrasonic Trigger 3
36	IO16	HCSR04_ECHO6	Ultrasonic Echo 6
37	IO26	HCSR04_ECHO3	Ultrasonic Echo 3
38	IO20	ESP_PI_SPI3_PM_DRDY1	SPI Data Ready 1
40	IO21	ESP_PI_SPI3_PM_DRDY2	SPI Data Ready 2

Table 8: Raspberry Pi 5 GPIO pin assignments

2.4.1 RPi5 Pin Function Categories

SPI3 (Controller Mode - Communication with ESP32):

- Pin 24 (CE0): Chip Select
- Pin 19 (COPI): Controller Out Peripheral In
- Pin 23 (SCLK): Serial Clock
- Pin 21 (CIPO): Controller In Peripheral Out
- Pin 38 (IO20): Data Ready 1 interrupt from ESP32
- Pin 40 (IO21): Data Ready 2 interrupt from ESP32

I2C (IMU and Stereo Cameras):

- Pin 3 (SDA): I2C Data
- Pin 5 (SCL): I2C Clock
- Pin 16 (IO23): IMU Reset
- Pin 18 (IO24): IMU Interrupt

UART (RPLiDAR C1):

- Pin 8 (TXD): LiDAR RX (Pi TX)
- Pin 10 (RXD): LiDAR TX (Pi RX)

Motor Driver Fault Lines:

- Pin 33 (IO13): Motor 1 nFAULT
- Pin 31 (IO6): Motor 2 nFAULT
- Pin 29 (IO5): Motor 3 nFAULT
- Pin 27 (ID_SD): Motor 4 nFAULT

HC-SR04 Ultrasonic Sensors (6 sensors):

- Sensor 1: Pin 7 (TRIG), Pin 11 (ECHO)
- Sensor 2: Pin 13 (TRIG), Pin 15 (ECHO)
- Sensor 3: Pin 35 (TRIG), Pin 37 (ECHO)
- Sensor 4: Pin 12 (TRIG), Pin 22 (ECHO)
- Sensor 5: Pin 26 (TRIG), Pin 28 (ECHO)
- Sensor 6: Pin 32 (TRIG), Pin 36 (ECHO)

2.5 Additional Peripheral Connections

2.5.1 IMU (Inertial Measurement Unit)

Hardware:

- BNO055: 9-axis absolute orientation sensor
- BMP280: Barometric pressure sensor

Protocol: I2C communication
Connection: Connects to RPi5 via Header J10:

- PI_SCL (Clock) routes to RPi5 Pin 5
- PI_SDA (Data) routes to RPi5 Pin 3
- IMU_RST routes to RPi5 Pin 16 (IO23)
- IMU_INT routes to RPi5 Pin 18 (IO24)

2.5.2 Stereo Cameras

Hardware: External stereo camera modules

Protocol: I2C communication

Connection: Connects via Header J12 to the same I2C bus (PI_SCL and PI_SDA), allowing the Pi to address them alongside the IMU using different I2C addresses.

2.5.3 Temperature Sensor

Hardware: DS18B20+ 1-Wire digital temperature sensor

Protocol: 1-Wire communication

Connection: Connected to ESP32-S3 IO48